

# Git & GitHub 입문 치트시트

처음 시작하는 사람을 위한 실전 명령어 정리

기초부터 첫 push까지

가장 먼저 알아둘 것. Git은 **commit**(내 PC에 저장) 과 **push**(GitHub 서버로 전송) 가 별개의 두 단계다. 커밋을 여러 번 쌓아두고 한 번에 push할 수 있다.

## 01 최초 1회 설정 (신원 등록)

커밋에는 "누가 했는지"가 기록되므로, 처음 한 번 내 신원을 알려줘야 한다. (안 하면 커밋 시 Author identity unknown 에러)

```
git config --global user.name "본인이름"
git config --global user.email "github가입이메일"
git config --global --list # 설정 확인
```

이메일은 GitHub 가입 이메일과 동일하게. 그래야 커밋이 GitHub 프로필·기여 그래프(잔디)에 연결된다. `--global` = 이 PC 전체 적용 (한 번만).

## 02 기존 프로젝트를 GitHub에 처음 올리기

오늘 진행한 전체 순서. 처음 등록만 이렇게 길고, 이후엔 3줄이면 된다(04번 참고).

| # | 명령어                                    | 의미                                         |
|---|----------------------------------------|--------------------------------------------|
| 1 | <code>git init</code>                  | 현재 폴더를 Git 저장소로 만들 ( <code>.git</code> 생성) |
| 2 | <code>.gitignore</code> 작성             | 올리지 않을 파일 지정                               |
| 3 | <code>git add .</code>                 | 변경분을 커밋 준비영역(staging)에 담기                  |
| 4 | <code>git commit -m "메시지"</code>       | 로컬에만 저장 ← 아직 서버 아님!                        |
| 5 | <code>git branch -M main</code>        | 기본 브랜치 이름을 main으로 (GitHub 관례)              |
| 6 | <code>git remote add origin URL</code> | GitHub 주소를 'origin' 별명으로 등록                |
| 7 | <code>git push -u origin main</code>   | 서버(GitHub)로 전송                             |

```
# 한 번에 복사해서 쓰는 전체 흐름
git init
# (.gitignore 파일 작성)
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin https://github.com/본인계정/저장소이름.git
git push -u origin main
```

GitHub 저장소는 "빈 깡통"으로 만들 것. New repository 시 README · .gitignore · License를 전부 체크 해제. 로컬에 이미 파일이 있으니 빈 상태여야 충돌이 없다. 위 6번의 주소는 본인이 만든 저장소 주소( `github.com/본인계정/저장소이름` )를 넣을 것.

**-u 의 효과:** push할 때 한 번 붙이면 "이 브랜치는 origin/main과 짝"으로 설정돼, 다음부터는 그냥 `git push` 만 쳐도 된다.

### 03 핵심 개념 정리

| 용어                           | 뜻                                                                   |
|------------------------------|---------------------------------------------------------------------|
| <b>staging</b><br>(add 한 상태) | 이번 커밋에 포함할 변경분을 담아두는 "장바구니". <code>add</code> 는 커밋할 때마다 매번 담는 동작이다. |
| <b>commit</b>                | 장바구니 내용을 하나의 버전으로 <b>내 PC에</b> 박제. 서버와 무관.                          |
| <b>push</b>                  | 로컬에 쌓인 커밋들을 GitHub로 전송하는 단계.                                        |
| <b>branch</b>                | 작업 "갈래"라는 개념(일반명사). 여러 작업 줄기를 나눠서 관리한다.                             |
| <b>main</b>                  | 기본 브랜치의 <b>이름</b> (고유명사). 과거엔 master. 기능상 특별할 건 없고 관례일 뿐.           |
| <b>origin</b>                | 원격 저장소(GitHub) 주소에 붙인 기본 별명.                                        |

"branch와 main의 차이?" 층위가 다른 비교다. branch = "파일"이라는 개념, main = "보고서.txt"라는 파일 이름. main은 여러 branch 중 하나의 이름이다.

### 04 일상 워크플로우 (앞으로 매번 이것만)

처음 등록 이후, 파일을 고치고 올릴 때 반복하는 3줄. 이게 가장 자주 쓰는 흐름이다.

```
git status          # 뭐가 바뀌었나 확인 (습관!)
git add .           # 변경분 담기
git commit -m "무엇을 바꿨는지" # 로컬 저장
git push            # GitHub로 전송 (-u 했으니 옵션 생략)
```

상태 색: 빨강 = 아직 안 담김 / 초록 = staging에 담김(커밋 대기)

### 05 자주 쓰는 명령어 모음

| 명령어                              | 설명                  |
|----------------------------------|---------------------|
| <code>git status</code>          | 현재 상태 확인 (가장 자주 씬)  |
| <code>git add .</code>           | 변경분 전체를 staging에 담기 |
| <code>git add 파일명</code>         | 특정 파일만 담기           |
| <code>git commit -m "메시지"</code> | 담긴 변경분을 로컬에 저장      |
| <code>git push</code>            | 로컬 커밋을 GitHub로 전송   |
| <code>git pull</code>            | GitHub의 최신 내용을 받아오기 |
| <code>git clone URL</code>       | 원격 저장소를 통째로 내려받기    |
| <code>git log --oneline</code>   | 커밋 기록을 한 줄씩 보기      |
| <code>git diff</code>            | 아직 안 담긴 변경 내용 보기    |
| <code>git restore 파일명</code>     | 수정한 파일을 직전 상태로 되돌리기 |
| <code>git branch</code>          | 브랜치 목록 보기           |
| <code>git switch -c 이름</code>    | 새 브랜치 만들고 이동        |

막히면 이 순서대로. 우선 `git status` 로 지금 상태를 확인하고 → 빨강/초록을 보고 → `add` · `commit` · `push` 중 어디서 멈췄는지 판단하면 된다.

## 06 오류 발생 시 대처

### ① Author identity unknown

신원 미설정. → 01번의 `user.name` / `user.email` 설정 후 다시 커밋.

### ② warning: LF will be replaced by CRLF

줄바꿈(개행) 방식 차이 알림일 뿐, **에러 아님**. 파일 안 깨진다. 무시하고 진행 OK.

### ③ fatal: unknown write failure on standard output

한글 출력 깨짐 문제(기능엔 무해). PowerShell 인코딩을 UTF-8로 맞추면 해결:

```
# PowerShell
chcp 65001
[Console]::OutputEncoding = [System.Text.Encoding]::UTF8
git config --global core.quotePath false
```

## 07 Windows / PowerShell 팁

터미널 종류에 따라 문법이 다르다. 리눅스식 `cat > 파일 << EOF` 는 Git Bash에선 되지만 PowerShell에선 에러. PowerShell에선 메모장이 가장 편하다.

.gitignore 만들기 (PowerShell)

```
# 방법 1: 메모장 (가장 쉬움)
notepad .gitignore

# 방법 2: PowerShell 명령
@"
node_modules/
.env
*.log
dist/
"@ | Out-File -FilePath .gitignore -Encoding utf8
```

**OneDrive 폴더 안에서 Git 쓰지 말 것.** OneDrive와 Git이 같은 `.git` 파일을 동시에 건드려 저장소가 깨질 수 있다. 작업 폴더는 OneDrive 밖, 한글·공백 없는 경로에 두자. (예: `D:\projects\my-app` )

## 08 다음에 배우면 좋은 것

- `git log --oneline --graph` — 커밋 히스토리 한눈에 보기
- `git restore` / `git reset` — 변경 되돌리기
- `git clone` / `git pull` — 다른 PC에서 받아와 작업하기
- `git switch -c feature/xxx` — 기능별 브랜치 나눠 작업하기
- `gh` CLI — 터미널에서 PR까지 처리

첫 push 성공 메시지: \* `[new branch] main → main`